
65C64 Architectural Manual

Instruction Reference

V0.1 Prelim

Copy_{cpy}

02 → 01

Copy the second operand into the first operand.

Typically the first operand (the destination) is a register and the second operand (the source) is any of the allowed operand types below. That can *load* a value in a memory location or a value in another register into the destination register.

Opcode Operand Types		
Operand Access Type (4 bits)		
Name	Mode	Type
Register / None	0	Register
Immediate	1	Constant
Embedded	15	
Address Absolute	2	Address
Address Absolute Register		
Address Absolute Indirect	3	
Address Absolute Register Indirect	4	
Address Absolute Indirect Register	5	
Address Offset	6	
Address Offset Register		
Address Offset Indirect	7	
Address Offset Register Indirect	8	
Address Offset Indirect Register	9	
Address Register	10	
Address Register Register	11	
Address Register Indirect	12	
Address Register Register Indirect	13	
Address Register Indirect Register	14	

Also fairly typically the first operand can be any of the allowed operand Address types above (a memory location) and the second operand is a register. That will *store* the register value into the memory location.

In some circumstances rather than a register a small value (mode 15, embedded) could also be stored into a memory location.

Register and Constant operand modes

Register mode 0

Register → Register

Copy a register into another register.

Format
cpy R₁, R₂
cpy R₁, R₂:s

Allowed registers for source register R₂: A, B, S, V, W, X or Y

Allowed registers for destination register R₁: A, B, S, V, W, X or Y

If the destination register is wider than the source register then the high bytes of the destination register are padded with zeros or sign extended.

If the destination register is narrower than the source register then the high bytes of the source register are truncated.

To use sign extension follow the source register with a colon and a small s.

Example

```
cpy A, B
cpy A, B:s
```

Immediate mode 1

Value → Register

Copy a value into a register.

Format
cpy R, val
cpy R, val:bit-width
cpy R, val:s

The value, val, immediately follows the *cpy* opcode in memory.

Allowed registers for destination register R: A, B, S, V, W, X or Y

If the register is wider than the immediate value then the high bytes of the register are padded with zeros or sign extended. A negative value also sets sign extension. To force sign extension follow the source register with a colon and a small s.

If the register is narrower than the immediate value then the high bytes of the immediate value are truncated.

The default bit-width of an immediate is 32 bits provided all the immediate bits fit. If more bits are needed then either 48 or 64 bits will be used by default.

If a wider immediate is required then append the bit-width to the end using a colon followed by 8, 16, 24, 32, 40, 48 or 64 (note 56 is not valid). If a small number in the range -16 to 16 is required then the bit-width must be included.

Example

```
cpy A, 8192           ;$00002000
cpy A, -32768          ;$ffff8000
cpy A, $1a2b3c4d       ;$1a2b3c4d
cpy A, $1a2b3c4d5e6f7081 ;$5e6f7081
cpy A, $4:s8           ;$04
cpy A, -1:16           ;$ffff
```

Embedded mode 15

Small Value → Register

Copy a small value into a register.

Format cpy R, val

The value, val, is embedded in the *cpy* opcode.

Allowed registers for destination register R: A, B, S, V, W, X or Y

Allowed values for val: -16 to -1, 0, 1 to 16.

Note the value 0 is from Register mode 0 (but that is only of interest for opcode decoding).

If the embedded value is negative then the sign is extended to the width of the register.

Example (A is set to 16 bits wide)

```
cpy A, 0 ;0000
```

```
cpy A, -8; ;fff8
```

```
cpy A, 7 ;0007
```

Address Absolute operand modes

Mode	Address Absolute		
	Absolute:	Address	
	Instruction in memory:	16bit Opcode	16bit Low
	Address calculation:	32bit High:Low = Effective Address	
2	Absolute Register:	Address + Register	
	Instruction in memory:	16bit Opcode	16bit Low
	Address calculation:	32bit High:Low + Register A, B, S, V, W, X or Y = Effective Address	
3	Absolute Indirect:	[Address]	
	Instruction in memory:	16bit Opcode	16bit Low
	Address calculation:	32bit [High:Low = Effective Address]	
4	Absolute Register Indirect:	[Address + Register]	
	Instruction in memory:	16bit Opcode	16bit Low
	Address calculation:	32bit High:Low + Register A, B, S, V, W, X or Y [Resultant Address = Effective Address]	
5	Absolute Indirect Register:	[Address] + Register	
	Instruction in memory:	16bit Opcode	16bit Low
	Address calculation:	32bit [High:Low + Register A, B, S, V, W, X or Y = Effective Address]	

Address Absolute mode 2

[Address] → Register

Copy a value in a memory address into a register.

```
Format
cpy R, [addr]
cpy R, [addr:32]
```

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.
Allowed registers for destination register R: A, B, S, V, W, X or Y

The width of the value in the source memory address will be the same as the destination register. If A is 16 bits wide then 16 bits will be loaded from memory, if A is 64 bits wide then 64 bits will be loaded from memory.

To force an Address Absolute address (rather than an Address Offset address) follow the address with a colon and the bit-width 32.

There is no sign extension as source and destination widths are always the same.

Example
 cpy A, [\$5e6f7081]
 cpy A, [0:32]
 cpy A, [\$5e6f:32]

Register → [Address]

Copy a register into a memory address.

```
Format
cpy [addr], R
cpy [addr:32], R
```

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.
Allowed registers for source register R: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be the same as the source register. If A is 16 bits wide then 16 bits will be stored into memory, if A is 64 bits wide then 64 bits will be stored into memory.

To force an Address Absolute address follow the address with a colon and the bit-width 32.

There is no sign extension as source and destination widths are always the same.

Example
cpy [\$5e6f7081], A
cpy [511:32], A

Small Value → [Address]

Copy a small value into a memory address.

```
Format
cpy [addr], val
cpy [addr:32], val
```

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.

Allowed values for val: -8 to -1, 0, 1 to 8; val is always 32 bits. No other widths are available.

To force an Address Absolute address follow the address with a colon and the bit-width 32.

Example
cpy [\$5e6f7081], 0
cpy [\$5e6f7081], -1
cpy [0:32], 0
cpy [2346:32], 5

Address Absolute Register mode 2

[Address + Register] → Register

Copy a value in a 'memory address plus a register' into a register.

```
Format
cpy R, [addr + RA]
cpy R, [addr:32 + RA]
```

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.

Allowed registers for destination register R: A, B, S, V, W, X or Y
Allowed registers for address register R_A: A, B, S, V, W, X or Y

The width of the value in the source memory address will be the same as the destination register.

To force an Address Absolute Register address follow the address with a colon and the bit-width 32.

Example
cpy A, [\$5e6f7081 + X]

Register → [Address + Register]

Copy a register into a memory address given by a 'memory address plus a register'.

```
Format
cpy [addr + RA], R
cpy [addr:32 + RA], R
```

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.

Allowed registers for source register R: A, B, S, V, W, X or Y
Allowed registers for address register R_A: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be taken from the width of the source register.

To force an Address Absolute Register address follow the address with a colon and the bit-width 32.

Example
cpy [\$5e6f7081 + X], A

Small Value → [Address + Register]

Copy a small value into a memory address given by a 'memory address plus a register'.

```
Format
cpy [addr + RA], val
cpy [addr:32 + RA], val
```

The 32 bit address follows the *cpy* opcode in memory.

Allowed registers for address register R_A: A, B, S, V, W, X or Y
Allowed values for val: -8 to -1, 0, 1 to 8; val is always 32 bits. No other widths are available.

To force an Address Absolute Register address follow the address with a colon and the bit-width 32.

Example
cpy [\$5e6f7081 + Y], 0
cpy [\$7081:32 + Y], -1
cpy [\$5e6f7081 + S], 1

Address Absolute Indirect mode 3

[[Address]] → Register

Copy a value in a memory address in another memory address into a register.
(For a full explanation see table: *Address Absolute* above)

```
Format
cpy R, [[addr]]
cpy R, [[addr:32]]
```

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.
The dereferenced address in addr is also always 32 bits.

Allowed registers for destination register R: A, B, S, V, W, X or Y

The width of the value in the source memory address will be the same as the destination register.

To force an Address Absolute Indirect address follow the address with a colon and the bit-width 32.

Example
cpy A, [[5e6f7081]]
cpy A, [[7081:32]]

Register → [[Address]]

Copy a value in a register into a memory address in another memory address.
(For a full explanation see table: *Address Absolute* above)

Format cpy [[addr]], R cpy [[addr:32]], R

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.
The dereferenced address in addr is also always 32 bits.

Allowed registers for source register R: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be the same as the source register.

To force an Address Absolute Indirect address follow the address with a colon and the bit-width 32.

Example
cpy [[$\$5e6f7081$]], A

Small Value → [[Address]]

Copy a small value into a memory address in another memory address.
(For a full explanation see table: *Address Absolute* above)

Format cpy [[addr]], val cpy [[addr:32]], val

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.
The dereferenced address in addr is also always 32 bits.

Allowed values for val: -8 to -1, 0, 1 to 8; val is always 32 bits. No other widths are available.

The width of the value in the destination memory address will be the same as the source register.

To force an Address Absolute Indirect address follow the address with a colon and the bit-width 32.

Example
cpy [[$\$5e6f7081$]], 1
cpy [[$\$7081:32$]], 8

Address Absolute Register Indirect mode 4

[[Address + Register]] → Register

Copy a value in a 'memory address plus a register' in another memory address into a register.
(For a full explanation see table: *Address Absolute* above)

Format cpy R, [[addr + R _A]] cpy R, [[addr:32 + R _A]]

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.

Allowed registers for destination register R: A, B, S, V, W, X or Y
Allowed registers for address register R_A: A, B, S, V, W, X or Y

The width of the value in the source memory address will be taken from the width of the destination register.

To force an Address Absolute Register Indirect address follow the address with a colon and the bit-width 32.

Example
cpy A, [[$\$1a2b3c4d + X$]]
cpy A, [[$\$3c4d:32 + B$]]

Register → [[Address + Register]]

Copy a value in a register into a 'memory address plus a register' in another memory address.
(For a full explanation see table: *Address Absolute* above)

Format cpy [[addr + R _A]], R cpy [[addr:32 + R _A]], R

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.

Allowed registers for source register R: A, B, S, V, W, X or Y
Allowed registers for address register R_A: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be taken from the width of the source register.

To force an Address Absolute Register Indirect address follow the address with a colon and the bit-width 32.

Example
cpy [[$\$5e6f7081 + X$]], A
cpy [[1:32 + B]], A

Small Value → [[Address + Register]]

Copy a value in a register into a 'memory address plus a register' in another memory address.
(For a full explanation see table: *Address Absolute* above)

Format cpy [[addr + R _A]], val cpy [[addr:32 + R _A]], val

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.

Allowed values for val: -8 to -1, 0, 1 to 8; val is always 32 bits. No other widths are available.
Allowed registers for address register R_A: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be taken from the width of the source register.

To force an Address Absolute Register Indirect address follow the address with a colon and the bit-width 32.

Example
cpy [[$\$5e6f7081 + X$]], 0
cpy [[1:32 + B]], 1

Address Absolute Indirect Register mode 5

[[Address] + Register] → Register

Copy a value in a memory address in another 'memory address plus a register' into a register.
(For a full explanation see table: *Address Absolute* above)

Format cpy R, [[addr] + R _A] cpy R, [[addr:32] + R _A]

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.
Allowed registers for destination register R: A, B, S, V, W, X or Y
Allowed registers for address register R_A: A, B, S, V, W, X or Y

The width of the value in the source memory address will be taken from the width of the destination register.

To force an Address Absolute Indirect Register address follow the address with a colon and the bit-width 32.

Example
cpy A, [[$\$1a2b3c4d5e6f7081 + X$]]
cpy [[33:32] + V], A

Register → [[Address] + Register]

Copy a value in a register into a memory address in another 'memory address plus a register'.
(For a full explanation see table: *Address Absolute* above)

Format cpy [[addr] + R _A], R cpy [[addr:32] + R _A], R

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.

Allowed registers for source register R: A, B, S, V, W, X or Y
Allowed registers for address register R_A: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be taken from the width of the source register.

To force an Address Absolute Indirect Register address follow the address with a colon and the bit-width 32.

Example
cpy [[$\$5e6f7081$] + X], A
cpy [[$\$5e6f7081:32$] + X], A

Small Value → [[Address] + Register]

Copy a value in a register into a memory address in another 'memory address plus a register'.
(For a full explanation see table: *Address Absolute* above)

Format cpy [[addr] + R _A], val cpy [[addr:32] + R _A], val

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.

Allowed values for val: -8 to -1, 0, 1 to 8; val is always 32 bits. No other widths are available.
Allowed registers for address register R_A: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be taken from the width of the source register.

To force an Address Absolute Indirect Register address follow the address with a colon and the bit-width 32.

Example
cpy [[$\$5e6f7081$] + X], 7
cpy [[$\$5e6f7081:32$] + X], -1

Address Offset operand modes

Mode	Address Offset
6	Offset: Address 16bit 16bit Instruction in memory: Opcode Low 32bit 0:Low = Effective Address
	Offset Register: Address + Register 16bit 16bit Instruction in memory: Opcode Low 32bit 0:Low + Register A, B, S, V, W, X or Y = Effective Address
	Offset Indirect: [Address] 16bit 16bit Instruction in memory: Opcode Low 32bit [High:Low] = Effective Address
8	Offset Register Indirect: [Address + Register] 16bit 16bit Instruction in memory: Opcode Low 32bit 0:Low + Register A, B, S, V, W, X or Y [Resultant Address] = Effective Address
	Offset Indirect Register: [Address] + Register 16bit 16bit Instruction in memory: Opcode Low 32bit [0:Low] + Register A, B, S, V, W, X or Y = Effective Address

Address Offset mode 6

[Offset] → Register

Copy a value in a memory address into a register.

Format
 cpy R, [offs]
 cpy R, [offs:16]

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.
 Allowed registers for destination register R: A, B, S, V, W, X or Y

The width of the value in the source memory address will be the same as the destination register. If A is 16 bits wide then 16 bits will be loaded from memory, if A is 64 bits wide then 64 bits will be loaded from memory.

To force an Address Offset Address (rather than an Address Absolute Address) follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

There is no sign extension as source and destination widths are always the same.

Example
 cpy A, [\$7081]
 cpy A, [0:16]
 cpy A, [\$5e6f:16]

Register → [Offset]

Copy a register into a memory address.

```
Format
cpy [offs], R
cpy [offs:16], R
```

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.
Allowed registers for source register R: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be the same as the source register. If A is 16 bits wide then 16 bits will be stored into memory, if A is 64 bits wide then 64 bits will be stored into memory.

To force an Address Offset Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

There is no sign extension as source and destination widths are always the same.

Example

```
cpy [$7081], A
cpy [511:16], A
cpy [$5e6f7081:16], A
```

Small Value → [Offset]

Copy a small value into a memory address.

```
Format
cpy [offs], val
cpy [offs:16], val
```

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.

Allowed values for val: -8 to -1, 0, 1 to 8; val is always 32 bits. No other widths are available.

To force an Address Offset Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

Example

```
cpy [$7081], 0
cpy [$7081], -1
cpy [0:16], 0
cpy [2346:16], 5
```

Address Offset Register mode 6

[Offset + Register] → Register

Copy a value in a 'memory address plus a register' into a register.

```
Format
cpy R, [offs + RA]
cpy R, [offs:16 + RA]
```

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.

Allowed registers for destination register R: A, B, S, V, W, X or Y
Allowed registers for Address register R_A: A, B, S, V, W, X or Y

The width of the value in the source memory address will be the same as the destination register.

To force an Address Offset Register Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

Example

```
cpy A, [$7081 + X]
cpy A, [$5e6f7081:16 + X]
```

Register → [Offset + Register]

Copy a register into a memory address given by a 'memory address plus a register'.

```
Format
cpy [offs + RA], R
cpy [offs:16 + RA], R
```

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.

Allowed registers for source register R: A, B, S, V, W, X or Y
Allowed registers for Address register R_A: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be taken from the width of the source register.

To force an Address Offset Register Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

Example
cpy [\$7081 + X], A

Small Value → [Offset + Register]

Copy a small value into a memory address given by a 'memory address plus a register'.

```
Format
cpy [offs + RA], val
cpy [offs:16 + RA], val
```

The 32 bit Address follows the *cpy* opcode in memory.

Allowed registers for Address register R_A: A, B, S, V, W, X or Y
Allowed values for val: -8 to -1, 0, 1 to 8; val is always 32 bits. No other widths are available.

To force an Address Offset Register Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

Example
cpy [\$7081 + Y], 0
cpy [\$7081:16 + Y], -1
cpy [\$7081:16 + S], 1

Address Offset Indirect mode 7

[[Offset]] → Register

Copy a value in a memory address in another memory address into a register.
(For a full explanation see table: *Address Offset* above)

```
Format
cpy R, [[offs]]
cpy R, [[offs:16]]
```

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.
The dereferenced Address in offs is also always 32 bits.

Allowed registers for destination register R: A, B, S, V, W, X or Y

The width of the value in the source memory address will be the same as the destination register.

To force an Address Offset Indirect Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

Example
cpy A, [[\$7081]]
cpy W, [[\$7081:16]]

Register → [[Offset]]

Copy a value in a register into a memory address in another memory address.
(For a full explanation see table: *Address Offset* above)

Format cpy [[offs]], R cpy [[offs:16]], R

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.
The dereferenced Address in offs is also always 32 bits.

Allowed registers for source register R: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be the same as the source register.

To force an Address Offset Indirect Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

Example
cpy [[$\$6FE4$]], A
cpy [[0:16]], A

Small Value → [[Offset]]

Copy a small value into a memory address in another memory address.
(For a full explanation see table: *Address Offset* above)

Format cpy [[offs]], val cpy [[offs:16]], val

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.
The dereferenced Address in offs is also always 32 bits.

Allowed values for val: -8 to -1, 0, 1 to 8; val is always 32 bits. No other widths are available.

The width of the value in the destination memory address will be the same as the source register.

To force an Address Offset Indirect Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

Example
cpy [[$\$6FE4$]], A
cpy [[77:16]], A

Address Offset Register Indirect mode 8

[[Offset + Register]] → Register

Copy a value in a 'memory address plus a register' in another memory address into a register.
(For a full explanation see table: *Address Offset* above)

Format cpy R, [[offs + R _A]] cpy R, [[offs:16 + R _A]]

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.

Allowed registers for destination register R: A, B, S, V, W, X or Y
Allowed registers for Address register R_A: A, B, S, V, W, X or Y

The width of the value in the source memory address will be taken from the width of the destination register.

To force an Address Offset Register Indirect Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

Example
cpy A, [[$\$1a2b + X$]]
cpy A, [[$\$3c4d:16 + B$]]

Register → [[Offset + Register]]

Copy a value in a register into a 'memory address plus a register' in another memory address.
(For a full explanation see table: *Address Offset* above)

Format cpy [[offs + R _A]], R cpy [[offs:16 + R _A]], R

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.

Allowed registers for source register R: A, B, S, V, W, X or Y
Allowed registers for Address register R_A: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be taken from the width of the source register.

To force an Address Offset Register Indirect Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

Example
cpy [[$\$7081 + X$]], A
cpy [[1:16 + B]], A

Small Value → [[Offset + Register]]

Copy a value in a register into a 'memory address plus a register' in another memory address.
(For a full explanation see table: *Address Offset* above)

Format cpy [[offs + R _A]], val cpy [[offs:16 + R _A]], val

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.

Allowed values for val: -8 to -1, 0, 1 to 8; val is always 32 bits. No other widths are available.
Allowed registers for Address register R_A: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be taken from the width of the source register.

To force an Address Offset Register Indirect Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

Example
cpy [[$\$7081 + X$]], 0
cpy [[1:16 + B]], 1

Address Offset Indirect Register mode 9

[[Offset] + Register] → Register

Copy a value in a memory address in another 'memory address plus a register' into a register.
(For a full explanation see table: *Address Offset* above)

Format cpy R, [[offs] + R _A] cpy R, [[offs:16] + R _A]

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.
Allowed registers for destination register R: A, B, S, V, W, X or Y
Allowed registers for Address register R_A: A, B, S, V, W, X or Y

The width of the value in the source memory address will be taken from the width of the destination register.

To force an Address Offset Indirect Register Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

Example
cpy A, [[$\$1a2b3c4d5e6f7081:16$] + X]
cpy [[33] + V], A

Register → [[Offset] + Register]

Copy a value in a register into a memory address in another 'memory address plus a register'.
(For a full explanation see table: *Addressing modes*)

Format cpy [[offs] + R _A], R cpy [[offs:16] + R _A], R

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.

Allowed registers for source register R: A, B, S, V, W, X or Y
Allowed registers for Address register R_A: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be taken from the width of the source register.

To force an Address Offset Indirect Register Address follow the offset with a colon and the bit-width 16. If the offset value is larger than 16 bits then the high bits are discarded.

Example
cpy [[$\$7081$] + X], A
cpy [[$\$7081:16$] + S], V

Small Value → [[Offset] + Register]

Copy a value in a register into a memory address in another 'memory address plus a register'.
(For a full explanation see table: *Addressing modes*)

Format cpy [[offs] + R _A], val cpy [[offs:16] + R _A], val

The offset, offs, is always 16 bits and follows the *cpy* opcode in memory.

Allowed values for val: -8 to -1, 0, 1 to 8; val is always 32 bits. No other widths are available.
Allowed registers for Address register R_A: A, B, S, V, W, X or Y

The width of the value in the destination memory address will be taken from the width of the source register.

To force an Address Offset Indirect Register Address follow the offset with a colon and the bit-width 16.

Example
cpy [[255] + X], 7
cpy [[$\$7081:16$] + X], -1

Address Register operand modes

Mode	Address Register	
10	Register:	Register 16bit
	Instruction in memory:	Opcode
11	Register Register:	Register + Register 16bit
	Instruction in memory:	Opcode
12	Register Indirect:	[Register] 16bit
	Instruction in memory:	Opcode
13	Register Register Indirect:	[Register + Register] 16bit
	Instruction in memory:	Opcode
14	Register Indirect Register:	[Register] + Register 16bit
	Instruction in memory:	Opcode

32bit	Register A, B, S, V, W, X or Y
=	Effective Address

32bit	Register A, B, S or V
+	Register V, W, X or Y
=	Effective Address

32bit	[Register A, B, S, V, W, X or Y]
=	Effective Address

32bit	Register A, B, S or V
+	Register V, W, X or Y
=	Resultant Address
	[Resultant Address]
=	Effective Address

32bit	Register A, B, S or V
+	Register V, W, X or Y
=	Effective Address

Address Register mode 10

[Register] → Register

Copy a value in a register memory address into a register.

Format
cpy R, [R_A]

Allowed registers for destination register R: A, B, S, V, W, X or Y

Allowed registers for address register R_A: A, B, S, V, W, X or Y

The width of the value in the source memory address will be the same as the destination register. If A is 16 bits wide then 16 bits will be loaded from memory, if A is 64 bits wide then 64 bits will be loaded from memory.

There is no sign extension as source and destination widths are always the same.

Example

cpy A, [W]

cpy X, [S]

Register → [Register]

Copy a register into a register given memory address.

Format
cpy [R_A], R

Allowed registers for source register R: A, B, S, V, W, X or Y

Allowed registers for address register R_A: A, B, S, V, W, X or Y

The width of the value in the source memory address will be the same as the source register.

There is no sign extension as source and destination widths are always the same.

Example

cpy [W], A
cpy [S], X

Small Value → [Register]

Copy a small value into a register given memory address.

Format
cpy [R_A], val

Allowed values for val: -8 to -1, 0, 1 to 8; val is always 32 bits. No other widths are available.

Allowed registers for address register R_A: A, B, S, V, W, X or Y

Example

cpy [A], 0
cpy [B], -1
cpy [S], 0
cpy [V], 5

Address Register Register mode 11

[Register + Register] → Register

Copy a value in a register memory address into a register.

Format
cpy R, [R_{A1}+R_{A2}]

Allowed registers for destination register R: A, B, S, V, W, X or Y

Allowed registers for first address register R_{A1}: A, B, S, or V

Allowed registers for second address register R_{A2}: V, W, X or Y

The width of the value in the source memory address will be the same as the destination register.

There is no sign extension as source and destination widths are always the same.

Example

cpy A, [V+X]
cpy B, [S+V]

Register → [Register + Register]

Copy a value in a register memory address into a register.

Format
cpy [R_{A1}+R_{A2}], R

Allowed registers for Source register R: A, B, S, V, W, X or Y

Allowed registers for first address register R_{A1}: A, B, S, or V

Allowed registers for second address register R_{A2}: V, W, X or Y

The width of the value in the source memory address will be the same as the source register.

There is no sign extension as source and destination widths are always the same.

Example

cpy [S+V], W

Address Register Indirect mode 12

[[Register]] → Register

Copy a value in a memory address in a register given memory address into a register.
(For a full explanation see table: *Address Register* above)

Format cpy R, [[R _A]]

The dereferenced address given by the register is always 32 bits.

Allowed registers for destination register R: A, B, S, V, W, X or Y

Allowed registers for address register R_A: A, B, S, V, W, X or Y

The width of the value in the source memory address will be the same as the destination register.

Example

cpy A, [[V]]

cpy A, [[S]]

Register → [[Register]]

Copy a value in a register into a memory address given by another memory address in a register.

(For a full explanation see table: *Address Register* above)

Format cpy [[R _A]], R

The dereferenced address given by the register is always 32 bits.

Allowed registers for source register R: A, B, S, V, W, X or Y

Allowed registers for address register R_A: A, B, S, V, W, X or Y

The width of the value in the source memory address will be the same as the source register.

The width of the value in the destination memory address will be the same as the source register.

Example

cpy [[\$5e6f7081]], A

Small Value → [[Register]]

Copy a small value into a memory address given by another memory address in a register.

(For a full explanation see table: *Address Register* above)

Format cpy [[R _A]], val
--

The address, addr, is always 32 bits and follows the *cpy* opcode in memory.

The dereferenced address in addr is also always 32 bits.

Allowed values for val: -8 to -1, 0, 1 to 8; val is always 32 bits. No other widths are available.

Example

cpy [[\$5e6f7081]], A

Address Register Register Indirect mode 13

$[[\text{Register} + \text{Register}]] \rightarrow \text{Register}$

Copy a value in a memory address in a 'register plus another register' given memory address into a register.

(For a full explanation see table: *Address Register* above)

Format
cpy R, $[[R_{A1} + R_{A2}]]$

Allowed registers for destination register R: A, B, S, V, W, X or Y

Allowed registers for first address register R_{A1} : A, B, S, or V

Allowed registers for second address register R_{A2} : V, W, X or Y

The width of the value in the source memory address will be the same as the destination register.

There is no sign extension as source and destination widths are always the same.

Example

cpy X, $[[A+W]]$

cpy V, $[[B+Y]]$

$\text{Register} \rightarrow [[\text{Register} + \text{Register}]]$

Copy a value in a register into a 'register plus another register' given memory address into a register.

(For a full explanation see table: *Address Register* above)

Format
cpy $[[R_{A1} + R_{A2}]]$, R

Allowed registers for Source register R: A, B, S, V, W, X or Y

Allowed registers for first address register R_{A1} : A, B, S, or V

Allowed registers for second address register R_{A2} : V, W, X or Y

The width of the value in the source memory address will be the same as the source register.

There is no sign extension as source and destination widths are always the same.

Example

cpy $[[V+X]]$, A

cpy $[[B+Y]]$, S

Address Register Indirect Register mode 14

$[[\text{Register}] + \text{Register}] \rightarrow \text{Register}$

Copy a value in 'a memory address in a register' plus another register given memory address into a register.

(For a full explanation see table: *Address Register* above)

Format
cpy R, $[[R_{A1}] + R_{A2}]$

Allowed registers for destination register R: A, B, S, V, W, X or Y

Allowed registers for first address register R_{A1} : A, B, S, or V

Allowed registers for second address register R_{A2} : V, W, X or Y

The width of the value in the source memory address will be the same as the destination register.

There is no sign extension as source and destination widths are always the same.

Example

cpy X, $[[A+W]]$

cpy V, $[[B+Y]]$

Register → [[Register] + Register]

Copy a value in a register into a 'register plus another register' given memory address into a register.

(For a full explanation see table: *Address Register* above)

Format

cpy [[$R_{A1} + R_{A2}$]], R

Allowed registers for Source register R: A, B, S, V, W, X or Y

Allowed registers for first address register R_{A1} : A, B, S, or V

Allowed registers for second address register R_{A2} : V, W, X or Y

The width of the value in the source memory address will be the same as the source register.

There is no sign extension as source and destination widths are always the same.

Example

cpy [[V+X]], A

cpy [[B+Y]], S